SIGNALS AND SYSTEMS

SW-CF

Four-Person Code Explanation and Viva Guide

Spatially-Weighted Chrominance Fusion

Complete explanations from the project discussion, compiled for all four team members. Includes every code block, worked example, evaluation script, and viva question from the four teaching responses.

Also included: the pretrained-model clarification, step-by-step running instructions, and cyan/orange quality-color explanation.

Study edition: 28 October 2026

> Evidence boundary: software checks and synthetic verification are complete. Real webcam-reference and real UBFC accuracy remain unmeasured. Pulse-interval variability is experimental, with no validated stress score.

Code fragments retain their teaching context. Some use ellipses or simplified notation; use the project source files when running or editing the application. Sample dates and output-folder names are illustrative.

Contents

Model clarification - pretrained face tracking and mathematical pulse estimation 3

Person 1 - Webcam acquisition, face tracking, and RGB extraction 5

Person 2 - Signal quality, POS projection, and spatial fusion 13

Person 3 - Filtering, BPM estimation, buffering, and interval variability 22

Person 4 - Integration, dashboard, exports, testing, and evaluation 31

Appendix A - Step-by-step running instructions 41

Appendix B - Cyan and orange face outlines 44

How to use this guide

Each member should understand the overall pipeline, study their assigned part, rehearse its one-minute explanation, and practise its viva questions. The original teaching responses are preserved rather than condensed. Shared files are divided by responsibility and function.

The model clarification is useful for everyone: MediaPipe locates facial landmarks; the pulse estimate comes from mathematical signal processing. No heart-rate model was trained.

Model clarification - pretrained face tracking and mathematical pulse estimation

I didn't train a heart-rate model. The project uses a **pretrained model to locate facial landmarks**, then uses mathematical signal processing to estimate pulse rate.

There are two separate jobs:

Job	How it works
Find facial regions	Google's pretrained MediaPipe face-landmarker model
Extract pulse and estimate BPM	Our RGB processing, POS projection, filtering, weighting, and spectral analysis

1. What does the pretrained model do?

MediaPipe receives a camera image and returns coordinates of points on the face.

We use those coordinates to select the forehead, cheeks, nose, and chin.

The model does not predict heart rate, detect blood flow, or estimate stress. It only helps us locate where to measure color.

We downloaded its existing model file, `face_landmarker.task`. We did not train or fine-tune it.

2. Where does the heart-rate information come from?

The webcam measures reflected light through its red, green, and blue channels.

Blood-volume changes associated with each heartbeat can produce very small changes in reflected skin color. Our goal is to extract that weak periodic component from much larger changes caused by lighting and movement.

For each facial patch, we average its RGB pixels repeatedly:

```
Frame 1 → average R, G, B
Frame 2 → average R, G, B
Frame 3 → average R, G, B
...
```

This turns images into **color signals over time**.

3. How does mathematics turn those signals into BPM?

The processing pipeline is:

```
RGB measurements over time
          ↓
Normalize color channels
          ↓
POS projection
Combine color variations to reduce some intensity interference
          ↓
Quality weighting
Give more influence to usable facial patches
          ↓
Bandpass filtering
Keep variations in the chosen pulse-frequency range
          ↓
Welch power spectrum
Find the strongest repeating frequency
          ↓
BPM = frequency in Hz × 60
```

For example, if the strongest accepted component is **1.2 Hz**:

```
1.2 repetitions per second × 60 = 72 BPM
```

These operations use equations and fixed rules. **They do not require training a neural network.**

4. Then what was UBFC supposed to be used for?

UBFC is intended for **evaluation**, not training in this implementation.

It contains face videos with synchronized reference pulse measurements. The evaluator can process those videos and compare our estimates against the reference.

We have not yet evaluated this project on the real UBFC dataset. The included synthetic tests check processing behavior using known artificial signals. They do not establish accuracy on real people.

5. Does that mean a webcam reading is definitely a heartbeat?

No. Movement or periodic lighting changes can also create a strong repeating color signal. Our quality checks help reject some failures, but cannot guarantee that every accepted peak is physiological.

That is why this is currently a **pulse-estimation prototype**, with real-reference validation still needed.

For your teacher, you can say:

> "Our project combines pretrained computer vision with classical signal processing. MediaPipe locates facial landmarks; it does not estimate heart rate. We extract average RGB time series from five facial regions, apply POS projection, quality-weighted fusion and filtering, then estimate pulse frequency using Welch PSD. No heart-rate model was trained. UBFC is intended to validate the estimates against reference measurements."

Person 1 - Webcam acquisition, face tracking, and RGB extraction

We'll divide the project into **four connected responsibilities**. Each person should understand the overall flow, then study their own section in depth.

Person	Responsibility	Main code
1	Webcam acquisition, face tracking, and extracting RGB measurements	`capture.py`, face tracking and patch extraction in `vision.py`
2	Signal quality, POS color projection, and spatial weighting	Quality calculation in `vision.py`; `resample()`, `pos()`, and fusion logic in `dsp.py`
3	Digital filtering, BPM estimation, pulse-interval variability, and buffering	Filtering, spectrum, and interval functions in `dsp.py`; `engine.py`, `config.py`
4	Application integration, dashboard, exports, testing, and evaluation	`main.py`, `app.py`, `dashboard.py`, `io.py`, `synthetic.py`, `benchmark.py`, tests and validation scripts

Some functions share a file because they belong to the same processing pipeline. **Divide ownership by function, rather than insisting that each person gets completely separate files.**

The overall connection is:

```
Person 1                  Person 2
Camera → face → RGB  →  quality → POS → weighted combination
                                              ↓
Person 4                  Person 3
Dashboard + validation ← filtering → BPM + variability
```

We'll cover **only Part 1 in detail now**.

PART 1 - Webcam acquisition, face tracking, and RGB extraction

What this person's section achieves

Your responsibility is to turn a webcam image into a small set of numerical measurements that the signal-processing code can use.

Your section takes this:

```
A color image containing a face
```

and produces this:

```
Timestamp
Five sets of average RGB values
Five facial polygons
```

The same extraction function also returns quality scores, whose calculation belongs to Part 2.

Your section does not calculate BPM. It supplies the measurements from which later sections estimate BPM.

The main files are capture.py and vision.py.

1. Opening the webcam

In `capture.py`, the `Camera` class manages camera access:

```
class Camera:
    def __init__(
        self, index=0, width=640, height=480,
        fps=30, backend="auto"
    ):
```

The arguments mean:

Argument	Meaning
`index=0`	Try the first camera
`width=640`	Request a frame width of 640 pixels
`height=480`	Request a frame height of 480 pixels
`fps=30`	Request 30 frames per second
`backend="auto"`	Select a suitable camera interface

A **backend** is the interface OpenCV uses to communicate with the operating system's camera system.

Our automatic selection prefers:

- **Windows:** DirectShow.
- **Linux:** V4L2.
- **Other systems:** OpenCV's default interface.

The camera is opened with:

```
cap = cv2.VideoCapture(index, api)
```

The code then checks:

```
if cap.isOpened():
```

This prevents the application from continuing as though the camera works when it failed to open.

What to tell the teacher:

> "We use OpenCV to access the webcam. We request 640 by 480 resolution at 30 FPS and choose a platform-appropriate capture backend. We check whether the device opened successfully before starting acquisition."

Important distinction: the code **requests** 30 FPS. It cannot force the hardware to deliver exactly 30 FPS.

2. Capturing frames and timestamps

The `_capture()` method repeatedly reads camera images:

```
ok, frame = self.cap.read()
stamp = time.perf_counter()
```

Here:

- `ok` indicates whether the read succeeded.
- `frame` is the image as a NumPy array.
- `stamp` records when the frame was delivered to our program.

For a 640 × 480 color image, the array normally has this shape:

```
(480, 640, 3)
```

That means:

```
480 rows × 640 columns × 3 color channels
```

We use `time.perf_counter()` because it is suitable for measuring elapsed time. These are **frame-delivery timestamps**, rather than exact camera exposure timestamps.

Why does timing matter?

Imagine a pulse pattern repeating once every second. To recognize its frequency correctly, the program needs to know **how much time passed between measurements**, not just how many images arrived.

What to tell the teacher:

> "Each frame receives a timestamp. Later processing uses the actual timing of the processed samples, because camera delivery and processing speed can vary."

3. Keeping the newest frame

The camera reads frames in a separate thread:

```
self.thread = threading.Thread(
    target=self._capture,
    daemon=True
)
self.thread.start()
```

A **thread** lets camera acquisition continue while the main application performs face tracking and signal processing.

We keep a single latest-frame slot:

```
self.latest = (self.sequence, stamp, frame)
```

A newer frame replaces the previous one.

Why?

Suppose the camera produces 30 frames per second but processing temporarily handles only 20. If every frame were queued, the dashboard could fall further behind the actual camera view.

Keeping the latest frame limits that application backlog. Some intermediate frames may be skipped.

The sequence number identifies whether a frame is new. A `threading.Condition` coordinates access so the processing code can wait for a fresh frame instead of repeatedly checking in a busy loop.

What to tell the teacher:

> "We use a separate capture thread with a single latest-frame slot. This prevents an increasing queue of old frames when processing falls behind. It reduces application backlog, but does not guarantee zero latency."

4. Finding the face

In `vision.py`, the `FaceTracker` class loads Google's pretrained face-landmarker model:

```
self.detector = (
    mp.tasks.vision.FaceLandmarker
    .create_from_options(options)
)
```

We configure it for one face:

```
num_faces=1
```

and timestamped video processing:

```
running_mode=mp.tasks.vision.RunningMode.VIDEO
```

For each image, we call:

```
result = self.detector.detect_for_video(
    image, millis
)
```

The result contains **facial landmarks**: numbered coordinates describing locations on the face.

Think of them as points attached to facial geometry. As the face moves, their coordinates change.

We use selected indices shared with the original 468-point face-mesh layout. **Do not say our current model necessarily returns exactly 468 points.**

We also make timestamps strictly increasing:

```
millis = max(
    self.last_ms + 1,
    int(timestamp * 1000)
)
```

That satisfies the video tracker's timestamp requirement.

What to tell the teacher:

> "We use a pretrained MediaPipe model to locate facial landmarks. We did not train a face detector. Our code uses selected landmark coordinates to construct the measurement regions."

5. Converting color order

OpenCV camera images use **BGR** channel order. MediaPipe expects **RGB** input.

Therefore, before tracking:

```
cv2.cvtColor(frame, cv2.COLOR_BGR2RGB)
```

This changes channel ordering; it does not change the underlying scene.

Remember:

```
OpenCV image:       Blue, Green, Red
Processing signals: Red, Green, Blue
```

Getting this wrong would feed incorrect channels into the later color projection.

6. Selecting five facial patches

The `PATCHES` variable stores landmark indices for five regions:

```
PATCHES = (
    (...),  # Forehead
    (...),  # Left cheek
    (...),  # Right cheek
    (...),  # Nose
    (...),  # Chin
)
```

The numbers identify landmarks. They are **not fixed pixel positions**.

MediaPipe gives normalized coordinates. We convert them into image coordinates:

```
scale = np.array([w, h])
points = landmarks[indices] * scale
```

For example, in a 640 × 480 image:

```
Normalized coordinate: (0.5, 0.25)

Pixel coordinate:
x = 0.5 × 640 = 320
y = 0.25 × 480 = 120
```

The patch positions therefore follow the tracked face instead of remaining fixed on the screen.

We shrink each patch toward its center:

```
center = points.mean(axis=0)
points = center + .80 * (points - center)
```

Each point's distance from the center becomes **80% of its original distance**. This helps keep measurements away from region boundaries.

Then:

```
polygon = cv2.convexHull(...)
```

constructs a convex boundary around the selected points.

What to tell the teacher:

> "We select five landmark-defined facial regions. Their coordinates update with the face. We shrink the polygons toward their centers to reduce contamination from surrounding boundaries."

This is geometric patch selection; it is not a pixel-by-pixel skin classifier.

7. Extracting only pixels inside a patch

A polygon is not usually rectangular. We use a **mask** to select its interior.

```
mask = np.zeros(..., np.uint8)

cv2.fillConvexPoly(
    mask,
    polygon - [x0, y0],
    255
)
```

The mask contains:

```
255 → inside the patch
0   → outside the patch
```

We select the interior pixels with:

```
pixels = frame[y0:y1, x0:x1][mask > 0][:, ::-1]
```

This line:

1. Crops the image to the polygon's bounding rectangle.
2. Keeps pixels whose mask is nonzero.
3. Reorders their channels from BGR to RGB.

The code rejects patches containing fewer than 25 pixels because they are too small for this implementation's measurement rule.

8. Averaging RGB values

For each valid patch:

```
mean = pixels.mean(axis=0)
```

Suppose three selected RGB pixels were:

```
[170, 130, 100]
[174, 132, 102]
[172, 128, 101]
```

Their average is:

```
[172, 130, 101]
```

Instead of sending thousands of pixel values to the DSP stage, we send three values per patch.

For five patches:

```
rgb.shape == (5, 3)
```

That is **15 color measurements per frame**.

Repeated over time, each channel becomes a discrete-time signal:

```
Forehead green:
G[0], G[1], G[2], G[3], ...
```

This is the key Signals and Systems connection in Part 1: we convert spatial image measurements into sampled time signals.

Spatial averaging can reduce some pixel-level noise. It does **not** eliminate disturbances affecting the whole patch, such as changing illumination.

9. Handling face loss

If the tracker finds no face:

```
if not result.face_landmarks:
    self.previous = self.previous_t = None
    return None
```

The application receives `None` and clears acquisition:

```
engine.reset(
    "No face - center your face and improve lighting"
)
```

This avoids continuing to display a previous measurement as though a face is still being observed.

10. Handoff to the other members

For each detected frame, extraction returns:

```
return (
    np.asarray(colors),
    np.asarray(qualities),
    polygons
)
```

Output	Used for
`colors`: `(5, 3)`	Input to pulse-signal extraction
`qualities`: `(5,)`	Quality gating and weighting, explained in Part 2
`polygons`	Drawing facial outlines on the dashboard
Timestamp from acquisition	Sampling-rate estimation and timing checks

A one-minute evaluation explanation for Person 1

"My section handles video acquisition and extracting facial color measurements. We use OpenCV to request webcam frames at 640 by 480 resolution and 30 FPS. Since actual timing can vary, we timestamp delivered frames. "Acquisition runs in a separate thread with a single latest-frame slot to prevent old frames from building up. We then use a pretrained MediaPipe face-landmarker model to locate the face. "Selected landmarks define five regions: forehead, both cheeks, nose, and chin. We convert the coordinates to pixels, construct smaller interior polygons, and use masks to select their pixels. "For every region, we calculate average red, green, and blue intensities. This gives fifteen color measurements per frame. Over time, these form discrete-time signals for the next stage. If the face disappears, we clear acquisition instead of retaining a stale result."

Viva questions for Person 1

Question	Answer
What is your input and output?	Input: timestamped camera frames. Output: five RGB means, facial polygons, and quality information for later processing.
Did you train the face detector?	No. We use a pretrained MediaPipe model and implement patch selection and measurement around its landmarks.
Why use landmarks instead of fixed rectangles?	Landmark-based regions follow changes in face position and scale. Fixed screen rectangles could start measuring the background.
Why use five regions?	They provide spatial redundancy. Later stages can reduce the influence of poorer regions. Five is our design choice, not a proven optimal number.
Does 30 FPS mean 30 heartbeats per second?	No. It means approximately 30 image samples per second. Heart rate is inferred from variation across those samples.
Is 30 FPS guaranteed?	No. We request it and use timestamps to account for actual processed timing.

Question	Answer
Why use a separate thread?	To continue acquisition while the main thread processes images.
Why discard old frames?	To limit application backlog and keep the displayed scene reasonably current.
Why average pixels?	It reduces the data size and some pixel-level noise while preserving shared temporal color variation.
Why convert BGR to RGB?	OpenCV and the tracker/processing code use different channel orders. Correct ordering is essential for later projection.
Does your section measure heart rate directly?	No. It extracts color measurements. Later stages estimate pulse frequency from their time variation.
What happens when the face disappears?	Tracking history is cleared and the application resets acquisition.

Question	Answer
What are the limitations of your section?	Poor light, occlusion, facial movement, small patches, and tracking errors can contaminate the measurements.

Before moving to Part 2, Person 1 should be able to explain this chain without reading:

```
Frame → timestamp → landmarks → polygons → masks → RGB means
```

Person 2 - Signal quality, POS projection, and spatial fusion

PERSON 2 - Signal quality, POS projection, and spatial fusion

Your responsibility is to answer:

> **"How do we extract useful color variations from the five facial regions, and decide which regions to trust more?"**

Person 1 supplies the facial RGB measurements. Your section converts them into pulse candidates and combines those candidates according to quality.

The relevant code is in vision.py and dsp.py.

```
Five RGB signals from Person 1
              ↓
Check lighting, clipping, and movement
              ↓
Resample onto uniform timestamps
              ↓
Apply POS separately to each region
              ↓
Assess spectral quality
              ↓
Normalize and combine using quality-based weights
              ↓
One combined pulse waveform
```

The filtering and spectral functions are shared with Person 3. Your explanation focuses on **how their outputs help select and combine regions**; Person 3 explains how those functions work.

1. Understand your input

For each frame, Person 1 produces:

```
rgb.shape == (5, 3)
```

For example:

```
Forehead:    [175, 132, 108]
Left cheek:  [170, 128, 104]
Right cheek: [178, 135, 110]
Nose:        [182, 140, 116]
Chin:        [165, 123, 101]
```

Each row contains average **red, green, and blue** values.

After collecting multiple frames:

```
rgb.shape == (N, 5, 3)
```

Here, `N` is the number of frames in the analysis window.

You now have five sets of RGB signals changing over time.

Your task is not to identify a heartbeat in a single image. You work with changes across a sequence of images.

2. Calculate physical image quality

This happens inside `extract_patches()` in `vision.py`.

We calculate a per-frame quality value, called `q`, for each patch. It lies between zero and one.

It considers three main problems.

A. Lighting

```
brightness = float(np.mean(pixels))
```

This measures the average brightness across the patch's pixels and channels.

The next expression creates a soft lighting score:

```
light = (
    np.clip((brightness - 20) / 40, 0, 1)
    * np.clip((240 - brightness) / 30, 0, 1)
)
```

You do not need to memorize every constant. Understand the behavior:

- Very dark patches receive a low score.
- Very bright patches receive a low score.
- Patches between those extremes receive a higher score.

These thresholds are **engineering choices**, not values learned from training data.

B. Clipping

```
clipping = float(
    np.mean(
        np.any(
            (pixels <= 8) | (pixels >= 247),
            axis=1
        )
    )
)
```

A pixel is flagged when any channel is near the black or white limit.

Why is clipping bad?

If a channel is already saturated near its maximum, a small increase in reflected light may not produce a corresponding recorded increase. Useful color variation can be lost.

The quality calculation penalizes clipping using:

```
(1 - clipping) ** 2
```

More clipped pixels therefore reduce quality.

C. Movement

The code compares current landmarks with previous landmarks:

```
motion = np.mean(
    np.linalg.norm(
        (landmarks[indices] - previous[indices]) * scale,
        axis=1
    )
)
```

It then normalizes that displacement:

```
motion /= face_scale * max(dt, 1e-3)
```

Here:

- `face_scale` represents face width in pixels.
- `dt` is the time between frames.

This expresses movement approximately in **face-widths per second**, instead of simply pixels per frame.

That helps account for differences in face size and frame timing.

This is landmark-based motion estimation. It is not optical flow.

The movement penalty is:

```
np.exp(-3 * motion)
```

More movement produces a smaller multiplier.

Combined physical quality

```
quality = (
    light
    * (1 - clipping)**2
    * np.exp(-3 * motion)
    * visible
)
```

A patch with good lighting, little clipping, and low movement gets a higher score. Invalid or out-of-frame patches receive zero quality.

What to tell the teacher:

> "We calculate a physical quality score for each region using lighting, pixel clipping, and normalized landmark movement. These are transparent heuristics that help identify unreliable measurements."

3. Resample measurements onto uniform timestamps

The `resample()` function in `dsp.py` handles uneven frame timing.

A webcam might deliver samples at:

```
0.000, 0.034, 0.066, 0.103 seconds
```

Digital filtering is easier to apply consistently when samples are equally spaced.

The code estimates the effective sampling rate:

```
fs = (len(times) - 1) / (times[-1] - times[0])
```

Then it creates an evenly spaced time grid:

```
grid = np.linspace(
    times[0], times[-1], len(times)
)
```

Each RGB channel and quality signal is interpolated onto that grid using:

```
np.interp(...)
```

What to tell the teacher:

> "Because webcam timing is not perfectly uniform, we use measured timestamps and linear interpolation to obtain uniformly spaced signals before DSP."

Important: interpolation estimates intermediate values. It does not recover information lost during missing frames.

4. Normalize each patch's RGB channels

Raw RGB values mostly represent average skin appearance and illumination. The pulse-related variation is much smaller.

Inside `pos()`, each channel is normalized over a local window:

```
c = (
    windows
    / np.maximum(
        windows.mean(axis=2, keepdims=True),
        EPS
    )
    - 1
)
```

The mathematical idea is:

```
normalized channel = channel / its temporal mean - 1
```

For example:

```
Green mean = 130
Current green = 130.65

Normalized variation = 130.65 / 130 - 1
                     = 0.005
```

That represents a **0.5% relative variation**.

Normalizing each patch separately reduces the influence of different average color levels between regions.

`EPS` is a tiny positive number used to prevent division by zero.

5. Apply POS projection

POS means Plane-Orthogonal-to-Skin. It is an established mathematical method, not a trained heart-rate model.

For normalized red, green, and blue signals, our code calculates:

```
s1 = c[:, 1] - c[:, 2]
s2 = c[:, 1] + c[:, 2] - 2*c[:, 0]
```

In simpler notation:

```
s1 = g - b
s2 = g + b - 2r
```

Why combine channels this way?

Consider an idealized disturbance that adds the same change, `d`, to all three normalized channels:

```
g - b → (g + d) - (b + d)
      → g - b
```

The common component cancels.

Similarly:

```
(g + d) + (b + d) - 2(r + d)
= g + b - 2r
```

Real lighting changes are more complicated, so this **reduces some common interference rather than removing all lighting effects**.

Next:

```
alpha = np.std(s1, axis=1) / np.maximum(
    np.std(s2, axis=1), EPS
)
```

And:

```
h = s1 + alpha[:, None] * s2
```

Mathematically:

```
alpha = std(s1) / std(s2)
h = s1 + alpha × s2
```

The standard-deviation ratio adjusts the relative scale of the two projected signals.

What to tell the teacher:

> "POS normalizes the color channels, forms two chrominance combinations, and combines them using a standard-deviation ratio. This helps suppress some intensity-related interference while retaining a pulse candidate."

Do not describe its output as guaranteed to be a clean heartbeat.

6. Use sliding windows and overlap-add

POS uses local windows lasting approximately **1.6 seconds**:

```
width = min(
    n, max(8, round(1.6 * fs))
)
```

At 30 samples per second:

```
1.6 × 30 ≈ 48 samples
```

The window moves forward one sample at a time.

```
Window 1: samples 0-47
Window 2: samples 1-48
Window 3: samples 2-49
```

Each window produces a local pulse candidate. Overlapping contributions are added together:

```
output[offset:offset+len(h)] += h[:, offset]
counts[offset:offset+len(h)] += 1
```

Then divided by the number of contributions:

```
return output / np.maximum(counts, 1)
```

This combines the overlapping local estimates without allowing samples covered by more windows to become larger merely because they received more contributions.

Remember the distinction:

Window	Purpose
Approximately **1.6 seconds**	Local POS normalization and projection
Approximately **6 seconds**	Main pulse-rate analysis window

7. Calculate the combined patch SQI

Physical image quality alone is insufficient. A well-lit, stationary patch might still contain a poor temporal signal.

After filtering each patch's POS output, the code calculates **spectral concentration**.

It asks:

> "How much of this patch's in-band spectral power lies near its strongest frequency?"

```
rho = power within ±0.18 Hz of the dominant frequency
      ------------------------------------------------
      total power within the allowed pulse band
```

A concentrated spectrum receives a higher value than a widely spread spectrum.

The combined patch score is:

```
patch_sqi = (
    np.mean(q, axis=0)
    * concentrations
    * valid
)
```

In words:

```
Patch SQI = average physical quality
            × spectral concentration
            × nonconstant-signal validity
```

`valid` prevents a nearly constant signal from receiving a useful score.

Keep these three quantities separate:

Quantity	Meaning
Physical quality, q	Current lighting, clipping, and movement
Patch SQI	Window-level combination of physical and spectral quality
Fusion weight	Final share of the combined waveform

The cyan/orange face outline uses **physical quality** q, not the complete patch SQI.

Spectral concentration is also **not the same as a measured signal-to-noise ratio**.

8. Normalize and weight the pulse candidates

Before combining regions, the code divides each waveform by its interior standard deviation:

```
normalized = waves / np.maximum(amplitude, EPS)
```

This prevents a patch from dominating simply because its waveform has a larger amplitude.

Then:

```
scores = patch_sqi ** 2
```

Squaring increases the contrast between higher and lower quality scores.

For example:

```
0.8² = 0.64
0.2² = 0.04
```

The code also applies the physical quality value at each sample:

```
weights_over_time = gates * scores
```

It normalizes these contributions so that, when usable contributions exist, the five weights sum to one:

```
w[k,n] = q[k,n] × SQI[k]²
         ------------------------------
         sum of all region contributions
```

This combines:

- **Window-level assessment:** is this region generally useful?
- **Per-sample assessment:** was this particular measurement physically usable?

These weights are applied during each analysis-window calculation.

9. Fuse the five regions

The final combination is:

```
fused = (
    normalized * weights_over_time
).sum(axis=1)
```

Mathematically:

```
fused[n] = w1[n] × pulse1[n]
         + w2[n] × pulse2[n]
         + w3[n] × pulse3[n]
         + w4[n] × pulse4[n]
         + w5[n] × pulse5[n]
```

That is **spatially weighted fusion**: combining measurements from different facial locations according to their quality.

The combined signal is filtered again to reduce unwanted variation introduced by changing weights. Person 3 will explain that filtering and the final BPM calculation.

The dashboard bars show the **time-average fusion weights over the analyzed window**.

A one-minute evaluation explanation for Person 2

"My section handles quality assessment, POS projection, and spatial fusion. We receive RGB time series from five facial regions. "First, we calculate physical quality using brightness, clipped pixels, and normalized landmark movement. We also resample the measurements onto uniform timestamps. "For each patch, POS normalizes the RGB channels and forms two chrominance combinations. These are combined using a standard-deviation ratio to produce a pulse candidate. "We then assess spectral concentration and combine it with physical quality to form a patch SQI. We standardize the candidate waveforms, square the SQI scores, apply per-sample quality gates, and normalize the resulting weights. "Finally, we calculate a weighted sum of the five candidates. This lets useful regions contribute more while poorer regions contribute less. These are mathematical rules, not a trained heart-rate model."

Viva questions for Person 2

Question	Answer
What is your main responsibility?	Extract pulse candidates from RGB signals and combine regions according to quality.
Did you train these weights?	No. They are calculated from explicit physical and spectral quality rules.
What does SQI mean?	Signal Quality Index. Here it combines average physical quality with spectral concentration.
Why not average all five regions equally?	A poor region could contaminate the combination. Weighting lets regions contribute differently.
What does POS stand for?	Plane-Orthogonal-to-Skin. It is an established color-projection method used for pulse extraction.
Why normalize RGB?	To express relative temporal variations rather than allowing average channel levels to dominate.

Question	Answer
Why use standard deviation in POS?	Its ratio adjusts the relative scale of the two projected components.
Why normalize waveform amplitudes before fusion?	To avoid confusing large amplitude with high quality.
Why square the SQI?	To emphasize quality differences. It is a design choice, not a proven optimal exponent.
Why use overlap-add?	To combine the local estimates produced by overlapping POS windows.
Is the entire system LTI?	No. POS scaling and fusion weights depend on the input, making the overall processing adaptive and nonlinear.
Does a strong spectral peak prove a heartbeat?	No. Periodic lighting or motion can also produce a concentrated spectrum.

Question	Answer
Does cyan mean medically healthy?	No. It only indicates that the current physical patch-quality score meets the display threshold.
What happens if every patch is unusable?	The weighting avoids division by zero, and the pipeline withholds an acceptable BPM rather than treating the result as reliable.
Does the simulation validate the quality estimator?	No. Its physical quality labels are supplied by the generator. It mainly tests the fusion behavior under those labels.

The chain Person 2 should remember is:

```
Quality → resampling → RGB normalization → POS
        → spectral SQI → normalized weights → fusion
```

Person 3 - Filtering, BPM estimation, buffering, and interval variability

PERSON 3 - Digital filtering, BPM estimation, buffering, and pulse-interval variability

Your section answers:

> **"Once we have pulse candidates from the facial regions, how do we clean the signal, estimate BPM, and decide whether the result is usable?"**

This is the section most directly connected to **Signals and Systems**.

Your main code is in:

- dsp.py: `bandpass()`, `spectrum()`, and `pulse_intervals()`.
- engine.py: buffering, update timing, and acquisition state.
- config.py: processing settings.

Your responsibilities overlap with Person 2 at filtering and spectral quality. Person 2 explains **how quality determines weights**; you explain **how filtering and frequency estimation work**.

```
Per-region POS signals
          ↓
Bandpass filtering
          ↓
Person 2's weighted fusion
          ↓
Final bandpass filtering
          ↓
Power spectrum → dominant frequency → BPM
          ↓
Quality acceptance / rejection

Separate longer history
          ↓
Peak detection → pulse intervals → experimental SDNN proxy
```

1. Understand the processing settings

In `config.py`:

```
fps: float = 30.0
window: float = 6.0
history: float = 65.0
hrv_seconds: float = 60.0
update: float = 0.5
low: float = 0.75
high: float = 2.5
```

Setting	Meaning
`fps = 30`	Requested camera rate; actual sample timing is measured
`window = 6`	Use approximately six seconds for each BPM analysis

Setting	Meaning
`history = 65`	Retain up to approximately 65 seconds of measurements
`hrv_seconds = 60`	Target duration for experimental variability analysis
`update = 0.5`	Recalculate short-window estimates approximately twice per second
`low = 0.75`	Lower bandpass cutoff in Hz
`high = 2.5`	Upper bandpass cutoff in Hz

Important: updating every half-second does not mean estimating heart rate from only half a second. Each update uses the latest six-second window.

2. Store measurements in a moving buffer

In `engine.py`, the `Engine` class manages the incoming measurements.

```
self.samples = deque(...)
```

A `deque` is a container that efficiently adds items at one end and removes them from the other.

Each stored item contains:

```
(timestamp, rgb_values, quality_scores)
```

New measurements are added:

```
self.samples.append(
    (timestamp, rgb.copy(), np.clip(quality, 0, 1))
)
```

Measurements older than the configured history are removed:

```
while (
    self.samples
    and timestamp - self.samples[0][0] > cfg.history
):
    self.samples.popleft()
```

From that longer history, we select the latest six seconds:

```
short = [
    s for s in self.samples
    if s[0] >= timestamp - cfg.window - 1e-6
]
```

For example:

```
At 10 seconds → analyze approximately seconds 4-10
At 10.5 seconds → analyze approximately seconds 4.5-10.5
At 11 seconds → analyze approximately seconds 5-11
```

This is a **sliding analysis window**.

What to tell the teacher:

> "We maintain a time-based buffer. Every half-second, we analyze the most recent six seconds for BPM, while retaining a longer history for pulse-interval variability."

3. Wait for enough data and check timing

Before six seconds have accumulated:

```
if elapsed < cfg.window:
    self.state.status = (
        f"Acquiring pulse: {elapsed:.1f} / {cfg.window:.0f} s"
    )
    return False
```

We need multiple pulse cycles to estimate periodicity. A single frame cannot provide pulse frequency.

The code also checks for capture gaps:

```
if gap > cfg.max_gap:
    self.reset("Capture gap - reacquiring")
```

The default maximum gap is **0.25 seconds**.

A large discontinuity makes a continuously sampled interpretation unreliable, so we reacquire rather than blindly join distant measurements.

The DSP also rejects effective sampling rates below **8 Hz**.

4. Remove baseline drift

The filtering function begins with:

```
signal.detrend(x, axis=0)
```

This subtracts a fitted linear trend.

Imagine a pulse signal whose baseline slowly rises because illumination changes:

```
Measured signal = periodic component + slow baseline change
```

Detrending reduces that straight-line baseline component before bandpass filtering.

It does **not** remove every possible lighting disturbance or movement artifact.

5. Apply the Butterworth bandpass filter

In `dsp.py`:

```
sos = signal.butter(
    4,
    [low, high],
    fs=fs,
    btype="bandpass",
    output="sos"
)
```

The passband is:

```
0.75-2.5 Hz
```

To convert Hz to BPM:

```
BPM = frequency × 60
```

Therefore:

```
0.75 × 60 = 45 BPM
2.5 × 60 = 150 BPM
```

The filter emphasizes variations within the project's chosen **45-150 BPM range** and attenuates slower and faster components.

For example:

Frequency	Equivalent rate	Intended treatment
0.2 Hz	12 repetitions/minute	Attenuate slow variation
1.2 Hz	72 BPM	Retain
5 Hz	300 repetitions/minute	Attenuate faster variation

A filter has a gradual transition, not an ideal brick-wall cutoff.

Why Butterworth?

A Butterworth design has a maximally flat magnitude response at the center of its passband. It gives a smooth response without passband ripple.

Why `output="sos"`?

SOS means **second-order sections**. The filter is represented as a cascade of smaller sections, which helps numerical stability.

The filter-order detail you should know

The `4` passed to SciPy is the **Butterworth prototype order**.

For this bandpass design:

- The resulting one-pass bandpass has **eight poles**.
- Forward/backward processing further changes the combined response.

Do not describe the complete operation simply as "a fourth-order filter" without this qualification.

6. Understand zero-phase filtering

The code applies:

```
signal.sosfiltfilt(
    sos,
    signal.detrend(x, axis=0),
    axis=0
)
```

This processes the buffered signal forward and backward.

The combined operation has zero phase, avoiding the phase shift of a one-direction filter. Its magnitude response is the square of the one-pass magnitude response.

Zero phase does not mean zero latency.

The program already has a buffered window available. It can process that stored window in both directions.

Consequently:

- Initial acquisition takes approximately six seconds.
- Window-edge samples can change as the window advances.
- This is not a strictly causal, zero-delay streaming filter.

The implementation excludes approximately **0.5 seconds at each end** when calculating spectral estimates to reduce boundary effects. This reduces, rather than guarantees removal of, edge distortion.

What to tell the teacher:

> "We use forward-backward filtering on buffered windows to avoid phase shift. This is compatible with a continuously updating dashboard, but it is not a causal zero-latency filter."

7. Estimate the power spectrum

The `spectrum()` function uses:

```
f, p = signal.welch(
    x,
    fs=fs,
    window="hann",
    nperseg=min(len(x), round(6 * fs)),
    nfft=next_fast_len(max(2048, len(x))),
    detrend="constant"
)
```

Its outputs are:

```
f → frequency values in Hz
p → estimated power spectral density at those frequencies
```

A time-domain waveform tells us how the signal varies with time.

A power spectrum tells us **which repetition frequencies are strongest**.

If the signal repeats about 1.2 times each second, we expect a strong component near **1.2 Hz**.

Why a Hann window?

A finite observation cuts the signal at its boundaries. The Hann window tapers the boundaries, reducing spectral leakage.

The tradeoff is a broader spectral peak.

An implementation detail worth knowing

Welch's method can average spectra from overlapping segments. However, our short analysis uses almost the entire available interior signal as one segment.

Therefore, **the short-window calculation may contain only one segment**. Do not claim it always averages many independent spectra.

8. Find the dominant frequency and calculate BPM

First, the code restricts the search:

```
keep = (f >= low) & (f <= high)
```

Then it finds the strongest in-band PSD value:

```
k = candidates[np.argmax(p[keep])]
```

The corresponding peak frequency is converted to BPM:

```
return f, p, float(60 * peak_hz), concentration
```

Example:

```
Dominant frequency = 1.5 Hz

Estimated BPM = 1.5 × 60
              = 90 BPM
```

The code also fits a small parabola around the strongest log-power bin to refine the peak location.

Zero padding and accuracy

The larger `nfft` produces a finer frequency grid. Peak interpolation can refine the estimate of an isolated peak.

Neither creates additional measurement information.

The rough frequency scale associated with a six-second observation is:

```
1 / 6 ≈ 0.167 Hz
```

or approximately:

```
0.167 × 60 ≈ 10 BPM
```

Our edge trimming shortens the spectral input further. This does not mean every result must be wrong by 10 BPM; it means that a finely displayed number is **not evidence of equally fine real-world resolving power or accuracy**.

9. Reject unreliable BPM estimates

After estimating the spectrum:

```
if (
    overall_q < config.min_quality
    or concentration < config.min_concentration
):
    bpm = None
```

The defaults are:

```
Minimum overall physical quality: 0.20
Minimum spectral concentration:   0.40
```

Spectral concentration measures how much in-band power lies near the dominant peak.

Other checks reject invalid timing or nearly constant signals.

When BPM is `None`, the dashboard displays no accepted numeric estimate.

What to tell the teacher:

> "We do not force a BPM output for every window. We withhold the estimate when physical quality or spectral concentration is inadequate."

These gates cannot prove that a strong periodic signal is physiological. Periodic lighting can still produce a misleading peak.

10. Calculate experimental pulse intervals

The longer-history analysis is handled by:

```
pulse_intervals(analysis, min_seconds=60.0)
```

The code waits until the analyzed history spans at least approximately **61 seconds**:

```
if t[-1] - t[0] < min_seconds + 1:
    return None
```

It then detects waveform peaks:

```
peaks, _ = signal.find_peaks(
    x,
    distance=max(1, int(fs * .40)),
    prominence=max(.35 * np.std(x), EPS)
)
```

Two important settings are:

Setting	Purpose
Minimum peak spacing of approximately 0.4 seconds	Reduce detection of implausibly close repeated peaks
Minimum prominence	Ignore small fluctuations that do not stand out sufficiently

Peaks near the boundaries are excluded. Their timing is refined with local parabolic interpolation.

If peak times are:

```
1.00 s, 1.82 s, 2.65 s, 3.46 s
```

the intervals are:

```
820 ms, 830 ms, 810 ms
```

In code:

```
intervals = np.diff(beat_t) * 1000
```

11. Screen intervals and calculate the SDNN proxy

Intervals are accepted only when they:

- Lie between approximately **400 and 1334 ms**.
- Differ from the median interval by no more than **20%**.

At least **90% of intervals** must pass. The median-interval rate must also agree reasonably with the spectral BPM.

The standard deviation is:

```
np.std(kept, ddof=1)
```

Mathematically:

```
SD = sqrt[
    sum((interval - mean_interval)²)
    / (number_of_intervals - 1)
]
```

A larger value means greater variation among the **accepted measured pulse intervals**.

Why call it a proxy?

Clinical SDNN concerns normal-to-normal beat intervals. Our webcam code detects and screens pulse peaks; it cannot reliably establish which beats are ECG-normal.

Also:

```
At 30 FPS:
time between samples ≈ 33.3 ms
```

Sub-frame interpolation refines a timing estimate, but does not restore missing temporal information.

Therefore, this is **experimental pulse-interval variability**, not a validated stress score. Screening may also remove genuine variation and bias the result.

The engine attempts long-history updates approximately every **five seconds**, once enough usable history exists.

A one-minute evaluation explanation for Person 3

"My section manages temporal buffering, digital filtering, pulse-rate estimation, and experimental interval variability. "We retain a moving history and analyze the latest six seconds approximately every half-second. We detrend the pulse candidates and use a Butterworth bandpass covering 0.75 to 2.5 Hz, equivalent to 45 to 150 BPM. "Filtering runs forward and backward on buffered windows to avoid phase shift. We then estimate the power spectrum, select its strongest accepted in-band frequency, and multiply by sixty to obtain BPM. "We withhold results when timing or signal quality is inadequate. A separate longer history supports peak detection and interval statistics. Its standard deviation is presented only as an experimental SDNN proxy, because webcam pulse timing is not equivalent to validated ECG-based HRV."

Viva questions for Person 3

Question	Answer
Why wait six seconds?	We need multiple cycles to estimate periodicity. Shorter windows give less frequency information; longer windows respond more slowly.
Why update every half-second?	To refresh the result while continuing to use the latest six-second window.
Are successive estimates independent?	No. Their windows overlap heavily, so they share most measurements.
Why use a bandpass filter?	To attenuate variations outside the chosen pulse-frequency range.
Why 0.75-2.5 Hz?	It corresponds to our selected 45-150 BPM demonstration range.
Can the system measure 180 BPM?	Not reliably with this configuration; that lies outside its chosen passband.

Question	Answer
Why Butterworth?	It provides a smooth magnitude response without passband ripple.
Is this an FIR or IIR design?	The underlying Butterworth design is IIR, represented using second-order sections.
Does zero phase mean zero delay?	No. Forward-backward processing operates on a stored window.
Why use a Hann window?	To reduce spectral leakage, with a tradeoff in peak width.
Does zero padding improve true resolution?	No. It gives a denser frequency grid without adding observation information.
How is BPM calculated?	Dominant accepted frequency in Hz multiplied by sixty.

Question	Answer
Why not count peaks for the main BPM?	Spectral estimation gives a dominant periodic rate without depending entirely on individual peak detections. Peaks are used separately for interval variability.
Why is no BPM sometimes displayed?	The input may fail timing, amplitude, physical-quality, or spectral-quality checks.
What does `ddof=1` do?	It uses the sample-standard-deviation denominator, `N−1`.
Does the SDNN proxy measure stress?	No validated stress mapping is implemented. It is an experimental statistic of screened pulse intervals.
What is a major limitation?	A strong periodic artifact can resemble pulse, and short windows plus low camera frame rates limit estimation.

Person 3's memory chain:

```
Buffer → detrend → bandpass → spectrum → peak frequency → BPM
                                          ↓
                              quality acceptance / rejection

Long history → waveform peaks → intervals → screening → SDNN proxy
```

Person 4 - Integration, dashboard, exports, testing, and evaluation

PERSON 4 - Application integration, dashboard, data export, testing, and evaluation

Your responsibility is to answer:

> **"How do all the processing modules become one usable application, and how do we check that it behaves correctly?"**

Persons 1-3 explain the measurement and DSP. You explain how those parts are connected, displayed, recorded, and evaluated.

Your main files are:

File	Responsibility
`main.py`	Starts the application
`app.py`	Selects the operating mode and coordinates everything
`dashboard.py`	Displays the camera, plots, weights, and readouts
`io.py`	Saves measurements and session information
`synthetic.py`	Generates artificial signals for demonstrations
`benchmark.py`	Compares estimates with UBFC reference measurements
`tests/` and `scripts/validate.py`	Automated checks and reproducible experiments

```
Choose input mode
        ↓
Get measurements
        ↓
Send them to the processing engine
        ↓
Receive analysis results
        ↓
Display results + save data
        ↓
Test behavior and evaluate errors
```

1. Starting the application

The entry point is main.py:

```
from swcf.app import main

if __name__ == "__main__":
    raise SystemExit(main())
```

This calls `main()` from `app.py`.

The condition means: execute this startup code when the file is run directly.

`SystemExit` returns the application's exit status to the operating system.

The Windows launchers simply run the appropriate command using the project's Python environment.

For example:

```
.\.venv\Scripts\python.exe main.py webcam
```

What to tell the teacher:

> "The entry point calls our application controller. The controller reads the selected mode, creates the necessary components, and runs the acquisition-processing-display loop."

2. Selecting the operating mode

In app.py, `argparse` reads command-line options.

The main commands are:

Command	Purpose
`webcam`	Process a live camera
`video`	Process a local video file
`demo`	Process generated RGB signals
`benchmark`	Evaluate local UBFC subject recordings
`doctor`	Print dependency versions and model status
`download-model`	Download the pretrained face-landmarker model

For example:

```
.\.venv\Scripts\python.exe main.py demo --bpm 90
```

selects generated input with a known **90 BPM** target.

This separation lets us test the DSP without needing a camera, while retaining the same core processing engine.

Important: the regular `demo` mode supplies generated RGB arrays directly. It does **not** recover those signals by running face tracking on the illustrated face.

The illustrated face is a visual representation of the simulation.

3. Connecting the modules

The `run()` function creates the main objects:

```
cfg = Config(window=args.window)
engine = Engine(cfg, mode=args.mode)
recorder = Recorder(out)
```

Their jobs are:

```
Config   → settings
Engine   → buffering and analysis
Recorder → saving measurements
```

For webcam mode, it also creates:

```
tracker = FaceTracker(args.model)
camera = Camera(...)
```

The application then repeatedly:

1. Gets a frame or generated sample.
2. Extracts patch measurements when using real video.
3. Saves the measurements.
4. Sends them to the engine.
5. Displays the current results.

The central processing call is:

```
updated = engine.push(t, rgb, q)
```

Here:

- `t` is the sample time.
- `rgb` contains five RGB means.
- `q` contains five physical quality scores.
- `updated` indicates whether a new analysis result was produced.

When an analysis update occurs:

```
recorder.estimate(engine.state)
```

saves its numeric results.

What to tell the teacher:

> "The application controller coordinates the modules. It passes timestamped patch measurements to the engine and uses the returned state for display and recording. The dashboard itself does not calculate BPM."

4. Handling live time and recorded-video time

These need different timing approaches.

For a webcam:

```
t = stamp - origin
```

Time comes from measured frame-delivery timestamps relative to the first frame.

For a recorded video:

```
t = index / video_fps
```

Time comes from the frame position and reported FPS.

Why does this matter?

If a ten-second recording is processed in three seconds, its pulse frequency must still be calculated using the recording's **ten-second media timeline**.

Otherwise, processing speed would incorrectly change the estimated BPM.

The current video implementation assumes **constant-frame-rate media with reliable FPS metadata**.

5. Rendering the dashboard

In dashboard.py, the main function is:

```
render(frame, polygons, quality, state, ...)
```

It constructs a dashboard image using OpenCV drawing functions.

For example:

```
cv2.putText(...)
cv2.polylines(...)
cv2.rectangle(...)
```

The application displays that image with:

```
cv2.imshow("SW-CF", image)
```

The interface includes:

Element	What it communicates
Camera image and polygons	Where measurements are taken
Cyan/orange outlines	Current physical quality of each region
Weight bars	Average contribution of each region in the analyzed window
BPM	Accepted spectral pulse-rate estimate
Pulse plot	Recovered waveform
PSD plot	Power across candidate pulse frequencies
Status message	Acquisition, tracking, face loss, or poor signal
SDNN proxy	Available experimental pulse-interval variability

The face-outline color is selected using:

```
color = CYAN if quality[p] >= .4 else ORANGE
```

That threshold concerns **physical image quality**, not health.

The BPM display uses:

```
bpm = (
    f"{a.bpm:.1f}"
    if a is not None and a.bpm is not None
    else "--"
)
```

This prevents an unavailable estimate from appearing as a valid number.

A plotting detail worth knowing: the plot helper rescales each displayed trace to fit its plotting area. The pulse plot is therefore useful for inspecting periodicity, but its displayed height is not an absolute physiological amplitude measurement.

6. Keyboard controls and shutdown

The application reads keyboard input through:

```
key = cv2.waitKey(1) & 255
```

Key	Action
Q or Escape	Quit
R	Reset acquisition history
S	Save a screenshot

Resource cleanup is placed in a `finally` block:

```
finally:
    if camera:
        camera.close()
    if video:
        video.release()
    if tracker:
        tracker.close()
```

The recorder is closed and GUI windows are destroyed there too.

A `finally` block is used because cleanup should run when processing finishes or an exception interrupts the run.

What to tell the teacher:

> "We release the camera, tracker, files, and windows during shutdown so resources are not left open after the application exits."

7. Saving measurements and results

The `Recorder` class in io.py creates the output files.

File	Contents
`rgb.csv`	Timestamp, fifteen patch-color values, and five quality values
`estimates.csv`	BPM, quality, concentration, sampling rate, available variability, and weights
`last_pulse.csv`	Final analyzed waveform, when available
`session.json`	Settings, source information, and final status
`dashboard.png`	Final dashboard image

For raw patch measurements:

```
self.raw.writerow([
    t,
    *np.asarray(rgb).ravel(),
    *quality
])
```

`ravel()` converts the five-by-three RGB array into fifteen values for a CSV row.

JSON stores structured metadata such as configuration and final status.

Why export intermediate data?

Because a screenshot alone cannot explain why the algorithm produced a result. Saved signals and quality values let us inspect failures and reproduce calculations.

The application does **not** record webcam video. Its saved dashboard screenshot does contain the camera image.

8. Generating synthetic test signals

In synthetic.py:

```
pulse = np.sin(2*np.pi*bpm/60*t)
```

For a target of 72 BPM:

```
72 / 60 = 1.2 Hz
```

The generator adds this known periodic variation to artificial RGB values.

It also introduces:

- Slow common illumination variation.
- Small random noise.
- Periodic interference in the nose region.
- Stronger random noise in the chin region.

The random generator uses a seed:

```
rng = np.random.default_rng(seed)
```

Using the same seed makes the experiment reproducible.

The key limitation

The simulator supplies low physical quality labels for its deliberately corrupted regions:

```
quality[:, 3] = .08
quality[:, 4] = .12
```

These are known labels supplied by the simulation, sometimes called **oracle annotations**.

Therefore:

> The simulation checks how processing and fusion behave when useful quality information is supplied. It does not prove that the webcam quality estimator identifies real artifacts correctly.

9. Comparing against baselines

The validation script compares three modes:

Mode	Main idea
`weighted`	POS candidates combined using quality-dependent weights
`equal`	POS candidates combined with equal contributions among valid regions

Mode	Main idea
`green`	Uses normalized green-channel signals instead of POS

The baselines help answer:

> "Does the more complicated method add value under the tested conditions?"

These modes share acquisition and rejection machinery. They are not completely independent implementations.

The synthetic experiment uses:

```
6 target rates
× 2 conditions: clean/corrupted
× 3 random seeds
= 36 generated signal cases
```

Each is processed by three modes, producing **108 case-mode evaluations**.

The files saved under `results/` contain the actual estimates and summaries.

Small errors on these artificial signals **must not be presented as real-person accuracy**.

10. Automated software checks

The saved test run reports:

```
18 passed
```

The tests cover behavior such as:

- Recovering known artificial pulse rates.
- Down-weighting deliberately corrupted regions.
- Rejecting constant or poor-quality input.
- Handling timestamp errors and capture gaps.
- Resetting after face loss.
- Checking filter behavior.
- Waiting before interval-variability calculations.
- Parsing ground truth and calculating coverage.
- Initializing the face model.
- Finishing a video without crashing at end-of-file.

Run them with:

```
.\.venv\Scripts\python.exe -m pytest -q
```

What to tell the teacher:

> "Our automated tests check software behavior and mathematical expectations under controlled inputs. Passing those tests is not equivalent to clinical validation or demonstrated accuracy on real subjects."

11. Evaluating against UBFC reference data

The evaluator in benchmark.py expects:

```
subject folder/
    vid.avi
    ground_truth.txt
```

It parses three ground-truth rows:

```
PPG waveform
Reference BPM
Reference timestamps
```

For the current BPM metric, it uses the **provided reference BPM**, averaged over the corresponding trailing six-second window.

It compares estimates approximately every half-second and records rejected or missing predictions.

Timing assumes the reference and video start together. The code does not optimize a time shift to obtain better scores.

Real UBFC evaluation has not yet been completed for this project. An artificial fixture tested that the evaluator runs, but that is not a real UBFC result.

12. Explain the evaluation metrics

Suppose:

```
Reference BPM:  70, 80, 90
Estimated BPM: 72, 77, 91
Errors:        +2, -3, +1
```

Mean Absolute Error - MAE

```
MAE = mean of absolute errors
    = (2 + 3 + 1) / 3
    = 2 BPM
```

Root Mean Square Error - RMSE

```
RMSE = sqrt(mean(error²))
     = sqrt((4 + 9 + 1) / 3)
     ≈ 2.16 BPM
```

RMSE gives more influence to larger errors.

Bias

```
Bias = mean signed error
     = (2 - 3 + 1) / 3
     = 0 BPM
```

Zero bias does not mean zero error: positive and negative errors can cancel.

Coverage

```
Coverage = accepted predictions / eligible evaluation windows
```

If the system accepts 80 of 100 eligible windows:

```
Coverage = 80%
```

Coverage matters because a system could obtain a low error by rejecting most difficult inputs.

Our evaluator therefore reports **error and coverage together**, along with per-subject outcomes and failures.

A one-minute evaluation explanation for Person 4

"My section integrates the application and handles visualization, recording, and evaluation. The controller selects webcam, video, or simulation mode, passes timestamped measurements to the processing engine, and displays the returned analysis state. "The OpenCV dashboard shows facial regions, quality colors, fusion weights, the pulse waveform, PSD, and accepted BPM. We also export measurements and metadata so the results can be inspected beyond screenshots. "For testing, we generate reproducible artificial RGB signals with known frequencies and compare weighted POS with equal POS and green-channel baselines. The saved automated run has eighteen passing tests. "We also provide a UBFC evaluator that reports MAE, RMSE, bias, and coverage. Real UBFC and webcam-reference accuracy remain unmeasured, so our completed evidence is software verification and synthetic validation."

Viva questions for Person 4

Question	Answer
Does the dashboard calculate BPM?	No. It displays the result produced by the engine and DSP functions.
Why have multiple input modes?	To support live demonstrations, reproducible recorded-video processing, and controlled artificial testing.
Does demo mode test face tracking?	No. It supplies generated RGB measurements directly. Face tracking needs separate integration checks.
Why use a fixed random seed?	To reproduce the same generated noise and compare methods consistently.
What does headless mean?	Processing without an interactive GUI window, while still saving outputs.
Why use video timestamps instead of processing time?	A recorded signal's frequency should not change when the computer processes it faster or slower.

Question	Answer
Why export RGB and quality values?	To inspect intermediate behavior, diagnose failures, and reproduce analysis.
Does the program save webcam video?	No, but the dashboard screenshots contain camera images.
What do 18 passing tests prove?	That the checked software behaviors passed those tests, not that every real-person measurement is accurate.
Was UBFC used for training?	No. The current algorithm uses fixed mathematical rules; UBFC is intended for evaluation.
Have real UBFC scores been measured?	Not yet. We must not label artificial-fixture results as real dataset performance.
Why compare baselines?	To assess whether added complexity improves performance under the tested conditions.

Question	Answer
Why report coverage?	Error alone hides how often the system refuses to provide a result.
Can bias be zero despite inaccurate estimates?	Yes. Positive and negative errors can cancel.
Are overlapping windows independent observations?	No. They share data, so window count is not the same as independent subject count.
What validation remains?	Real reference comparisons across subjects and conditions, including errors, rejection coverage, and variability reliability.

Person 4's memory chain:

```
Select mode → connect modules → display state → export data
            → test known inputs → compare with references
```

Appendix A - Step-by-step running instructions

Everything is already installed in your project folder on this laptop, so you can start directly.

1. Open the project folder

Open File Explorer and paste this into its address bar:

```
C:\Users\somas\OneDrive\Documents\Signals and Systems\sns
```

You should see files such as `main.py`, `run_demo.bat`, and `run_webcam.bat`.

You do **not** need to extract the ZIP on this laptop. That ZIP is for submitting or transferring the project.

2. Run the simulation first

Double-click `run_demo.bat`.

A dashboard should open with an illustrated face. This uses generated color signals with a known **72 BPM** pulse, so it works without accessing your camera.

Wait about **six seconds**. You should see:

- **Heart rate:** approximately 72 BPM.
- **Region weights:** the forehead and cheeks contribute more because the simulation deliberately corrupts the nose and chin signals.
- **Pulse waveform:** the recovered periodic signal.
- **Welch PSD:** a frequency plot whose main peak determines BPM.

The simulation runs for approximately **70 seconds**, then closes. You can also press **Q** to quit.

What you're running: `run_demo.bat` launches the project's Python environment and executes `main.py demo`. This checks that the dashboard and processing pipeline work; it does not measure a person.

3. Run your actual webcam

Close the simulation, then double-click `run_webcam.bat`.

For the best chance of getting a usable signal:

1. Close other applications using the camera.
2. Sit facing the camera with steady light illuminating your face.
3. Keep your whole face visible.
4. Stay still and avoid talking.
5. Wait at least **six seconds**.

The dashboard should outline five facial regions and begin estimating BPM when the signal passes its quality checks.

Display	Meaning
Five outlined regions	Areas where the software averages skin color

Display	Meaning
Region weights	Each region's contribution to the combined signal
BPM	Estimated pulse rate from the dominant spectral frequency
Pulse waveform	Extracted signal in normalized, arbitrary units
Welch PSD	Distribution of signal power across pulse-rate frequencies
SDNN proxy	Experimental variability of detected pulse intervals

The SDNN proxy needs **at least about 61 seconds of continuously acceptable input**. It may remain unavailable when quality is insufficient. It is **not a validated stress measurement**.

4. Use the keyboard controls

Click the dashboard window so it has keyboard focus.

Key	Action
Q or **Escape**	Stop the application
R	Clear the signal history and start acquisition again
S	Save a dashboard screenshot

Press **R** when changing subjects or significantly changing lighting.

5. Find your saved results

Open the project's `outputs` folder.

Each run creates a folder such as:

```
outputs\webcam_20261027_...
```

Inside it:

File	What it contains
`dashboard.png`	Final dashboard screenshot, including the camera image
`rgb.csv`	Recorded average RGB values and quality scores for the five patches
`estimates.csv`	BPM estimates, weights, quality, and available variability estimates
`last_pulse.csv`	Final analyzed pulse waveform
`session.json`	Configuration and final session information

The program saves numeric measurements and screenshots; it does **not** record a webcam video.

6. Understand the processing sequence

The webcam mode performs this sequence:

```
Camera frames
    ↓
Locate facial landmarks
    ↓
Select forehead, cheeks, nose, and chin
    ↓
Average RGB pixels within each region
    ↓
Assess lighting, clipping, and movement
    ↓
Use POS projection to extract pulse-related color variation
    ↓
Filter and combine regions according to quality
    ↓
Find the dominant frequency using Welch PSD
    ↓
Convert frequency to BPM and display the results
```

For example, a dominant frequency of **1.2 Hz** means:

```
1.2 cycles/second × 60 = 72 BPM
```

The filter keeps approximately **0.75-2.5 Hz**, corresponding to the project's chosen **45-150 BPM** range.

7. Run commands manually when troubleshooting

If a launcher closes unexpectedly, open PowerShell and run:

```
cd "C:\Users\somas\OneDrive\Documents\Signals and Systems\sns"
```

Check the environment:

```
.\.venv\Scripts\python.exe main.py doctor
```

Run the simulation:

```
.\.venv\Scripts\python.exe main.py demo
```

Run the webcam:

```
.\.venv\Scripts\python.exe main.py webcam
```

Running this way keeps error messages visible.

If the webcam cannot open, try:

```
.\.venv\Scripts\python.exe main.py webcam --backend msmf
```

If you have another camera, try:

```
.\.venv\Scripts\python.exe main.py webcam --camera 1
```

8. Run the automated checks

From that same PowerShell window:

```
.\.venv\Scripts\python.exe -m pytest -q
```

This checks the processing against known artificial signals, verifies rejection/reset behavior, and exercises parts of the video and evaluation paths. The completed run had **18 passing tests**.

For your first session, do **simulation → webcam → inspect the output folder**. Leave UBFC benchmarking until those steps work; it requires downloading the actual dataset separately.

Appendix B - Cyan and orange face outlines

The face outlines show each region's current image quality:

- **Cyan:** quality score **≥ 0.4** - the patch has more usable lighting and less movement.
- **Orange:** quality score **< 0.4** - the patch may be moving, shaded, too dark, or overexposed.

Orange patches generally contribute less to the pulse estimate. The **region-weight bars** show their actual contribution, which also considers signal quality over time.

These colors indicate **measurement quality**, not your health or stress level.